Go-reforged

Tu as terminé go-reloaded. Ton outil sait convertir des nombres, changer la casse des mots et remettre la ponctuation en ordre. Il est maintenant temps de le transformer en véritable outil : robuste, rapide, testé, et capable de traiter des textes que tu n'as pas prévus.

Objectifs

Dans ce projet, tu vas reprendre go-reloaded et le reconstruire sur des bases solides. Ton programme ne sera plus une suite de `strings.Replace` : il devra lire le texte, le découper en éléments (tokens), appliquer des règles dans un ordre précis, signaler les erreurs avec leur position exacte, et traiter des fichiers de plusieurs centaines de mégaoctets sans exploser la mémoire.

Comme pour go-reloaded, le projet sera corrigé par des auditeurs, qui seront d'autres étudiants. Tu seras toi aussi auditeur. Les cas limites sont nombreux : prépare tes propres fichiers de test, pour toi et pour les projets que tu corrigeras.

Prérequis : avoir validé go-reloaded. Go 1.18 ou supérieur (nécessaire pour le fuzzing).

Introduction

- Ton projet doit être écrit en **Go**.
- Le code doit respecter les **bonnes pratiques**.
- Les **tests unitaires** sont **obligatoires** (voir la partie 7), ainsi qu'un test de fuzzing et des benchmarks.
- Seuls les packages de la bibliothèque standard de Go sont autorisés.
- Le programme ne doit **jamais** paniquer, quelle que soit l'entrée.

Architecture imposée

Ton code doit être organisé en plusieurs packages, chacun avec une responsabilité unique :

Package	Rôle
`lexer`	Transforme une ligne de texte en une liste de tokens, avec leur ligne et colonne
`transform`	Contient les règles de transformation
`render`	Reconstruit le texte final à partir des tokens
`main`	Lit les arguments, orchestre le traitement, gère les erreurs et les codes retour

Chaque règle de transformation doit implémenter une interface commune, par exemple :

```
type Rule interface {
    Name() string
    Apply(tokens []Token) ([]Token, []Diagnostic)
}
```

Ajouter une nouvelle règle ne doit pas nécessiter de modifier les règles existantes.

Ordre de traitement

Pour que tous les projets produisent le même résultat, les étapes doivent être appliquées **dans cet ordre**, ligne par ligne :

1. Analyse lexicale (échappements, modificateurs, balises, mots, ponctuation)
2. Blocs (partie 2.4)
3. Modificateurs postfixés, de gauche à droite (parties 1 et 2)
4. Articles (partie 4)
5. Ponctuation, guillemets et parenthèses (partie 3)
6. Majuscule en début de phrase, seulement si `--sentence-case` est activé (partie 3.5)
7. Écriture du résultat (partie 5.4)

Partie 1 : Compatibilité avec go-reloaded

Toutes les règles de go-reloaded doivent continuer à fonctionner exactement comme avant : `(hex)`, `(bin)`, `(up)`, `(low)`, `(cap)`, leurs versions avec nombre `(up, <number>)`, la ponctuation, les apostrophes `' '` et la règle `a` → `an`.

Tous les exemples du sujet de go-reloaded doivent donner le même résultat avec go-reforged (sauf si une option modifie volontairement le comportement).

Définition d'un mot : un mot est un token qui n'est ni un signe de ponctuation, ni un guillemet, ni un modificateur, ni une balise de bloc. Les nombres sont des mots. Quand un modificateur compte `n` mots, il ignore tout ce qui n'est pas un mot. (Ex : "hello, world (up, 2)" -> "HELLO, WORLD")

Partie 2 : Nouveaux modificateurs

2.1 Conversions numériques

Modificateur	Effet	Exemple
`(oct)`	Nombre octal → décimal	"17 (oct)" -> "15"
`(to-hex)`	Nombre décimal → hexadécimal, en majuscules	"255 (to-hex)" -> "FF"
`(to-bin)`	Nombre décimal → binaire	"5 (to-bin)" -> "101"
`(roman)`	Chiffre romain valide → décimal	"MMXXVI (roman)" -> "2026"
`(to-roman)`	Nombre décimal entre 1 et 3999 → chiffre romain	"1984 (to-roman)" -> "MCMLXXXIV"

Règles communes à toutes les conversions (y compris `(hex)` et `(bin)`) :

- Elles acceptent un nombre : `(hex, 3)` convertit les 3 mots précédents.
- `(hex)` ne tient pas compte de la casse : "ff" et "FF" donnent tous les deux "255".
- Un signe `-` en tête est accepté : "-1A (hex)" -> "-26".

- Un nombre qui dépasse la capacité d'un `int64` est invalide.
- `(roman)` refuse les chiffres romains mal formés, comme "IIII" ou "VX".

2.2 Nouveaux modificateurs de texte

- `(rev)` inverse les lettres du mot précédent. Les caractères accentués doivent être conservés intacts. (Ex : "noël (rev)" -> "lëon")
- `(title)` met une majuscule à chaque mot, **sauf** aux petits mots `a`, `an`, `the`, `of`, `and`, `or`, `in`, `on`, `at`, `to`, qui restent en minuscules. Le premier mot concerné prend toujours une majuscule. (Ex : "the lord of the rings (title, 5)" -> "The Lord of the Rings")
- `(up)`, `(low)`, `(cap)` et `(title)` doivent fonctionner avec les lettres accentuées. (Ex : "élan (cap) et ÉTÉ (low)" -> "Élan et été")

`(rev)` et `(title)` acceptent aussi un nombre : `(rev, 2)`, `(title, 4)`.

2.3 Chaînage et échappement

- Plusieurs modificateurs à la suite s'appliquent au même mot, de gauche à droite. (Ex : "FF (hex) (to-bin)" -> "11111111")
- Un modificateur précédé d'une barre oblique inverse n'est pas appliqué : il est écrit tel quel, sans la barre. (Ex : "Type \(up) to shout (up)" -> "Type (up) to SHOUT")
- Un texte entre parenthèses qui n'est pas un modificateur connu reste du texte normal. (Ex : "the (hello) word" -> "the (hello) word")

2.4 Blocs

Un bloc applique une transformation à tous les mots qu'il contient. Les blocs disponibles sont `[up]...[/up]`, `[low]...[/low]`, `[cap]...[/cap]` et `[title]...[/title]`.

- Les balises disparaissent du texte final.
- Les blocs peuvent être imbriqués. Le bloc **le plus intérieur l'emporte**.
- Les blocs sont appliqués **avant** les modificateurs postfixés, qui ont donc le dernier mot.
- Un bloc doit être fermé sur la même ligne que celle où il est ouvert.

Exemples :

- "[up]this is [low]VERY[/low] important[/up]" -> "THIS IS very IMPORTANT"
- "[low]Hello World (up)[/low]" -> "hello WORLD"

2.5 Cas d'erreur des modificateurs

Dans tous les cas suivants, le programme **laisse le modificateur tel quel dans le texte**, ne modifie pas le mot, et affiche un avertissement (voir partie 5.3) :

- le mot ne correspond pas à la conversion demandée (ex : "ZZ (hex)", "3 (bin)") ;
- le nombre du modificateur est nul, négatif ou n'est pas un nombre (ex : `(up, 0)`, `(up, -1)`, `(up, two)`) ;

- `(to-roman)` reçoit un nombre hors de l'intervalle 1–3999 ;
- il n'y a aucun mot avant le modificateur sur la ligne.

Si le nombre demandé dépasse le nombre de mots disponibles sur la ligne, le modificateur s'applique à tous les mots disponibles et un avertissement est affiché. **Un modificateur ne traverse jamais un saut de ligne.**

Un bloc non fermé, fermé par la mauvaise balise ou fermé sans avoir été ouvert est une **erreur** (pas un simple avertissement) : voir la partie 5.3.

Partie 3 : Ponctuation et guillemets avancés

3.1 Ponctuation

Les règles de go-reloaded s'appliquent toujours : `.`, `,`, `!`, `?`, `:` et `;` sont collés au mot précédent et séparés du suivant par un espace. Toute suite de ces signes forme un groupe qui reste collé (`...`, `!?`, `?!...`).

3.2 Guillemets simples et doubles

- Les guillemets doubles `"` suivent la même règle que les apostrophes `'` de go-reloaded : ils se collent aux mots qu'ils entourent.
- Les guillemets peuvent être imbriqués. (Ex : `He said: " she told me ' no ' "` -> `He said: "she told me 'no'"`)
- Un guillemet sans partenaire sur la même ligne est laissé tel quel et produit un avertissement.

3.3 Apostrophes de contraction

Une apostrophe placée **entre deux lettres** n'est pas un guillemet : c'est une contraction ou un nom propre, et elle ne doit pas être appariée. (Ex : "' don't do it '" -> "'don't do it'", "O'Neil" reste "O'Neil")

3.4 Parenthèses

Les parenthèses qui ne forment pas un modificateur se collent au texte qu'elles contiennent. (Ex : "it was (almost) perfect" -> "it was (almost) perfect")

3.5 Majuscule en début de phrase (option)

Si l'option `--sentence-case` est activée, la première lettre du texte et la première lettre après `.`, `!` ou `?` passent en majuscule.

- Cette règle ne fait que mettre des majuscules : elle ne met jamais de lettre en minuscule.
- Elle ne touche pas un mot qui a été explicitement modifié par un modificateur ou un bloc.
- Elle s'applique aussi d'une ligne à la suivante : si une ligne se termine par `.`, la ligne suivante commence par une majuscule.

Partie 4 : Articles

- Comme dans go-reloaded, `a` devient `an` si le mot suivant commence par une voyelle ou un `h`.

- **Nouveau :** la règle inverse s'applique aussi. `an` devient `a` si le mot suivant ne commence ni par une voyelle ni par un `h`. (Ex : "an banana" -> "a banana")
- La casse de l'article est conservée : `A` -> `An`, `An` -> `A`, `AN` -> `A`.
- Les articles sont traités **après** les modificateurs. (Ex : "a ORANGE (low)" -> "an orange")
- Un mot qui commence par un chiffre ne déclenche aucun changement.

Fichier d'exceptions

L'option `--articles <file>` charge un fichier d'exceptions. Chaque ligne contient un mot et l'article qu'il exige, séparés par un espace. Les exceptions sont prioritaires sur la règle générale et ne tiennent pas compte de la casse.

```
$ cat articles.txt
university a
house a
```

Une ligne mal formée dans ce fichier est une erreur (voir partie 5.3).

Partie 5 : Ligne de commande

5.1 Arguments

```
$ go run . [options] <input> <output>
```

- `-` à la place de `<input>` signifie l'entrée standard, et `-` à la place de `<output>` signifie la sortie standard.
- Le fichier d'entrée et le fichier de sortie peuvent être le même fichier.
- Les sauts de ligne de l'entrée doivent être conservés, y compris les lignes vides et les fins de ligne Windows (`\r\n`).

5.2 Options

Les options doivent être gérées avec le package `flag`.

Option	Effet
`--sentence-case`	Active la règle de la partie 3.5
`--articles <file>`	Charge un fichier d'exceptions pour les articles
`--strict`	Tout avertissement devient une erreur : aucun fichier de sortie n'est écrit
`--check`	N'écrit rien, affiche seulement les avertissements et les erreurs (`<output>` est alors facultatif)
`--workers <n>`	Nombre de goroutines utilisées pour le traitement (par défaut : nombre de cœurs)

5.3 Avertissements, erreurs et codes de sortie

Les avertissements et les erreurs sont écrits sur la **sortie d'erreur** (`stderr`), dans ce format exact :

```
<file>:<line>:<column>: <warning|error>: <message>
```

- La ligne et la colonne commencent à 1. La colonne est comptée en **caractères** (runes), pas en octets.
- La position indiquée est celle du modificateur, de la balise ou du guillemet en cause.
- Pour l'entrée standard, `<file>` vaut `<stdin>`.

Code	Signification
`0`	Succès (des avertissements ont pu être affichés)
`1`	Erreur d'utilisation ou de fichier (arguments manquants, fichier introuvable, droits insuffisants)
`2`	Erreur de contenu (bloc mal formé, fichier d'articles invalide, avertissement en mode `--strict`)
`3`	En mode `--check` uniquement : des avertissements ont été trouvés

En cas de code `1` ou `2`, le fichier de sortie ne doit être **ni créé ni modifié**.

5.4 Écriture sûre

Le fichier de sortie ne doit jamais se retrouver à moitié écrit, même si le programme est interrompu ou si une erreur survient pendant le traitement. Écris d'abord dans un fichier temporaire situé dans le même dossier, puis renomme-le à la place du fichier final.

Partie 6 : Performance et concurrence

- Le programme doit traiter des fichiers de **plus de 500 Mo**. Sa consommation mémoire ne doit pas dépendre de la taille du fichier : le fichier ne doit jamais être chargé entièrement en mémoire.
- Une seule ligne peut faire jusqu'à **10 Mo** (attention à la limite par défaut de `bufio.Scanner`).
- Avec `--workers <n>`, les lignes sont traitées en parallèle, mais le résultat doit être **strictement identique**, dans le même ordre, à un traitement avec `--workers 1`.
- Sur une machine à plusieurs cœurs, `--workers 4` doit être sensiblement plus rapide que `--workers 1` sur un gros fichier.
- Fournis un petit programme `tools/gen/main.go` qui génère un fichier de test de la taille demandée : `go run ./tools/gen 500MB big.txt`.

Partie 7 : Tests

- Chaque règle doit avoir ses propres tests unitaires, écrits sous forme de **tests table-driven**.
- La couverture doit être d'au moins **80 %** : `go test -cover ./...`
- Fournis un test de fuzzing `FuzzProcess` qui vérifie au minimum que le programme ne panique jamais : `go test -fuzz=FuzzProcess ./...`
- Pour une entrée qui ne contient ni `\`, ni modificateur invalide, ni guillemet orphelin, le traitement doit être **idempotent** : traiter deux fois le texte donne le même résultat que le traiter une fois. Ton test de fuzzing doit vérifier cette propriété.

- Fournis des benchmarks pour le traitement complet d'un fichier : `go test -bench=. ./...`

Utilisation

```
$ cat sample.txt
The answer is FF (hex) (to-bin) and the year is MMXXVI (roman) .
$ go run . sample.txt result.txt
$ cat result.txt
The answer is 11111111 and the year is 2026.
$ cat sample.txt
[up]this is [low]VERY[/low] important[/up] , trust me .
$ go run . sample.txt result.txt
$ cat result.txt
THIS IS very IMPORTANT, trust me.
$ cat sample.txt
She said : " don't worry , it's ' fine ' " and left .
$ go run . sample.txt result.txt
$ cat result.txt
She said: "don't worry, it's 'fine'" and left.
$ cat sample.txt
I ate an banana and a orange in a hour .
$ go run . sample.txt result.txt
$ cat result.txt
I ate a banana and an orange in an hour.
$ cat sample.txt
It is a university next to an house .
$ go run . sample.txt result.txt
$ cat result.txt
It is an university next to an house.
$ go run . --articles articles.txt sample.txt result.txt
$ cat result.txt
It is a university next to a house.
$ cat sample.txt
Type \(up) to shout (up) . the lord of the rings (title, 5) is long .
$ go run . --sentence-case sample.txt result.txt
$ cat result.txt
Type (up) to SHOUT. The Lord of the Rings is long.
$ cat sample.txt
Add ZZ (hex) and 3 (bin) please (up, -1) .
$ go run . sample.txt result.txt
sample.txt:1:8: warning: invalid hexadecimal number "ZZ"
sample.txt:1:20: warning: invalid binary number "3"
sample.txt:1:33: warning: invalid count in modifier "(up, -1)"
$ echo $?
0
$ cat result.txt
Add ZZ (hex) and 3 (bin) please (up, -1).
$ go run . --strict sample.txt result2.txt
sample.txt:1:8: error: invalid hexadecimal number "ZZ"
sample.txt:1:20: error: invalid binary number "3"
sample.txt:1:33: error: invalid count in modifier "(up, -1)"
$ echo $?
2
$ ls result2.txt
ls: cannot access 'result2.txt': No such file or directory
$ echo "hello (up) [cap]big world[/cap]" | go run . - -
HELLO Big World
```

Bonus

- `--diff` : affiche les lignes modifiées, avant et après, au lieu d'écrire le fichier.
- `--explain` : affiche sur `stderr` chaque transformation appliquée, avec sa position.
- Règle d'élision française activable avec `--lang fr` : "le arbre" -> "l'arbre", "de eau" -> "d'eau".

Ce projet t'aidera à découvrir

- L'analyse lexicale (lexer) et la représentation d'un texte sous forme de tokens
- Les interfaces Go et la conception de code extensible
- Le package `bufio` et le traitement en flux (streaming) de gros fichiers
- Les goroutines, les channels et la préservation de l'ordre dans un traitement concurrent
- Le package `flag`, les codes de sortie et la séparation `stdout` / `stderr`
- L'écriture atomique de fichiers avec l'API du système de fichiers (**fs**, **os**)
- Les runes et la gestion de l'Unicode en Go
- Les tests table-driven, le fuzzing et les benchmarks